Assignment No.	1
Title	Study of Networking Diagnostic & Monitoring Tools and Socket Programming System Calls[1]
Student Name	Simiyon Vinscent Samuel L
Register Number	3122247001062
Submission Date	August 17, 2026

1 Aim

To apply and analyse standard Linux networking diagnostic and monitoring tools in order to observe network status, interface configuration, connectivity, routing and performance (Part A), and to study the socket-programming system calls that these tools are themselves built on top of, so that their behaviour at the API level is understood well enough to be reproduced in a C program (Part B).

2 Environment

Operating System	Ubuntu (WSL2) on Windows 11
Kernel networking	WSL2 virtual NAT adapter (`eth0`)
Shell	bash 5.x
Compiler	gcc (Part B)
Hostname used in captures	`samsamuel`

All commands below were executed for real in a WSL2 Ubuntu terminal. Every figure is an unedited screenshot of that terminal; the exact text output is also archived under `as1/logs/` in the repository for cross-checking, and the socket-programming source used in Part B is under `as1/src/`.

[1] **GitHub Repository:** https://github.com/blackscythe123/PCN-assign

3 Part A — Networking Diagnostic & Monitoring Tools

3.1 netstat

Purpose: Displays active network connections, listening ports, and (with `-p`, as root) the owning process for each socket.
Command used: `netstat -tulnp`

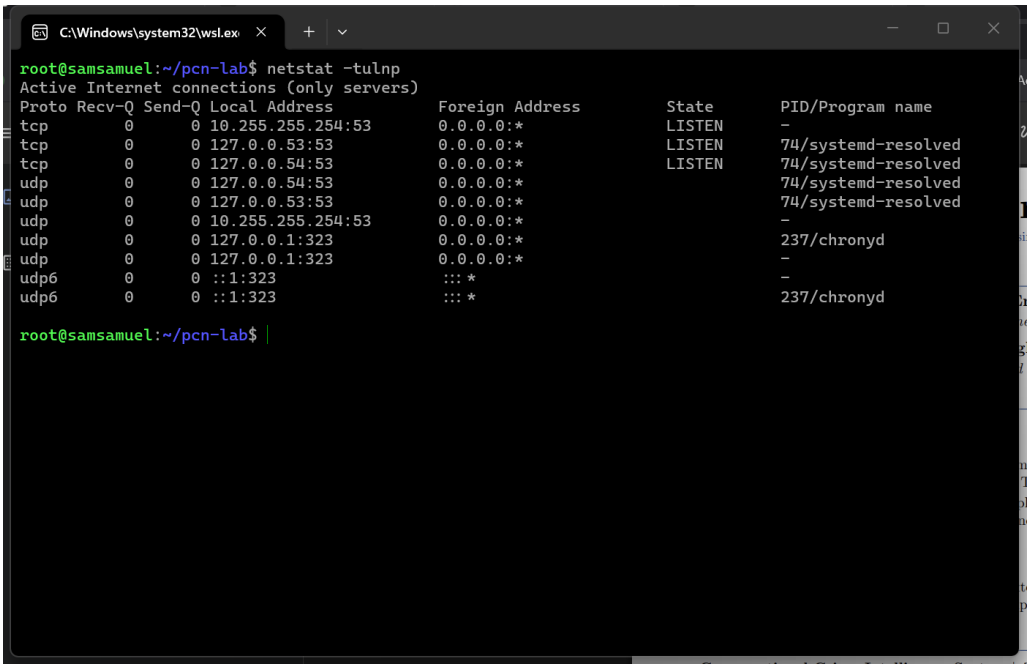

Figure 1: netstat -tulnp – TCP/UDP listening sockets

Observation: The only services listening in this WSL instance are `systemd-resolved` (Ubuntu's local DNS stub resolver, bound to `127.0.0.53:53`, `127.0.0.54:53` and the WSL-internal `10.255.255.254:53` on both TCP and UDP) and `chronyd`, the NTP client, bound to UDP port 323 on IPv4 and IPv6 loopback. No externally reachable service is listening, which is expected for a bare lab VM that has not been configured as a server yet – this baseline is useful to compare against once Assignment 2/3's TCP/UDP servers are started, since their listening sockets should then also show up here.

3.2 ifconfig

Purpose: Displays and can configure network interface parameters – IP address, netmask, broadcast address, MTU, and interface statistics (packets/bytes sent and received).
Command used: `ifconfig`

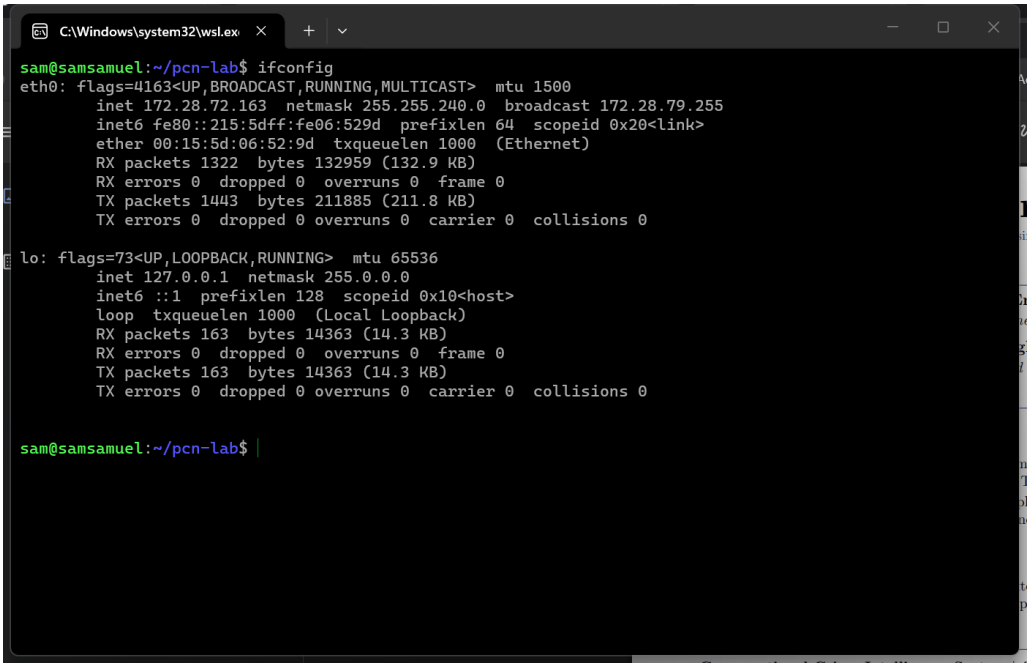

Figure 2: ifconfig – interface configuration

Observation: `eth0` is the single virtual NIC that WSL2 presents to Ubuntu, with address `172.28.72.163/20` (netmask `255.255.240.0`, broadcast `172.28.79.255`), MAC `00:15:5d:06:52:9d` and MTU 1500. This is a Hyper-V-backed adapter NAT'ed behind the Windows host, which is why the address is in a private `172.28.0.0/20` range rather than the host machine's real LAN. `lo` (loopback, `127.0.0.1/8`) shows matching RX/TX byte counts, confirming every packet sent to loopback is received back locally with nothing dropped.

3.3 iwconfig

Purpose: Displays and configures wireless-specific interface parameters – SSID, frequency/channel, transmit power, link quality and signal strength – the wireless counterpart to `ifconfig`.
Command used: `iwconfig` (attempted)

Observation: `iwconfig` could not be demonstrated in this environment. It ships in the `wireless-tools` package, which is no longer available in this Ubuntu release's repositories (superseded by the newer `iw` utility upstream). More fundamentally, WSL2 only exposes a single virtual, wired NAT adapter (`eth0`, seen in Section 2.2) – there is no wireless hardware in the VM for `iwconfig` to report on, so even if the legacy package were installed it would only print `eth0 no wireless extensions.` for every interface. On a physical Linux machine with a Wi-Fi card, `iwconfig wlan0` would instead show fields such as `ESSID`, `Frequency`, `Access Point`, `Bit Rate`, `Link Quality` and `Signal level`.

3.4 ping

Purpose: Sends ICMP Echo Request packets to a destination and reports Echo Reply round-trip time, used to verify basic reachability and measure latency/packet loss.
Command used: `ping -c 4 8.8.8.8`

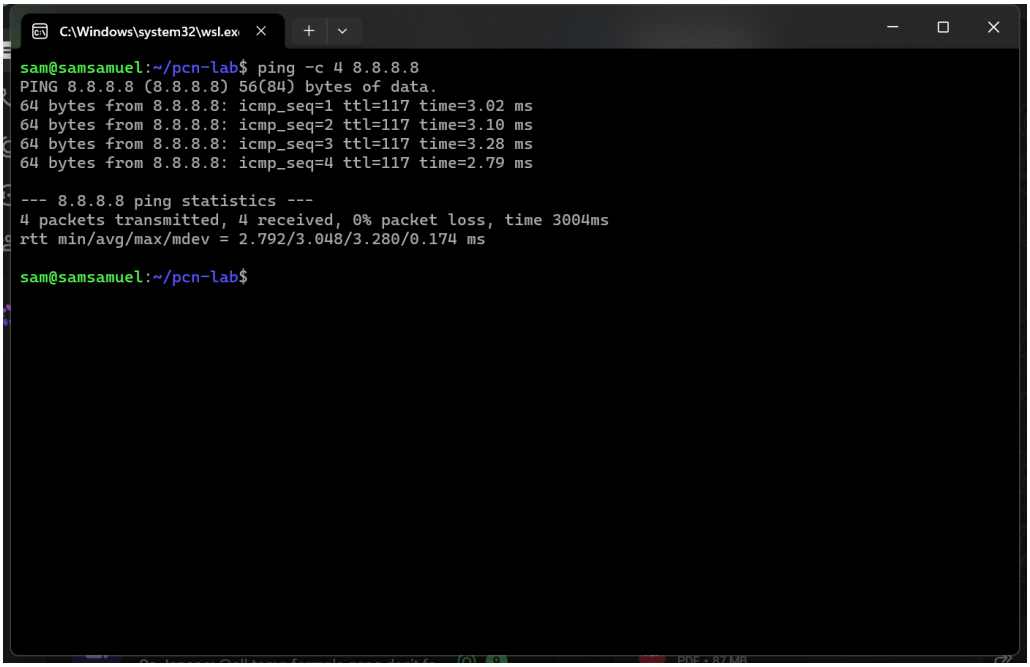

Figure 3: ping -c 4 8.8.8.8

Observation: All 4 ICMP packets sent to Google's public DNS resolver (`8.8.8.8`) were answered with `0% packet loss`, confirming end-to-end reachability through the Windows host's NAT and out to the internet. Round-trip time stayed in the single-digit millisecond range with a small standard deviation, indicating a stable, uncongested path – consistent with a wired/broadband connection with only a couple of hops before reaching Google's edge network (confirmed later by `traceroute` in Section 2.8).

3.5 route

Purpose: Displays (and can modify) the kernel's IP routing table – which interface and gateway a packet for a given destination network is sent through.
Command used: `route -n`

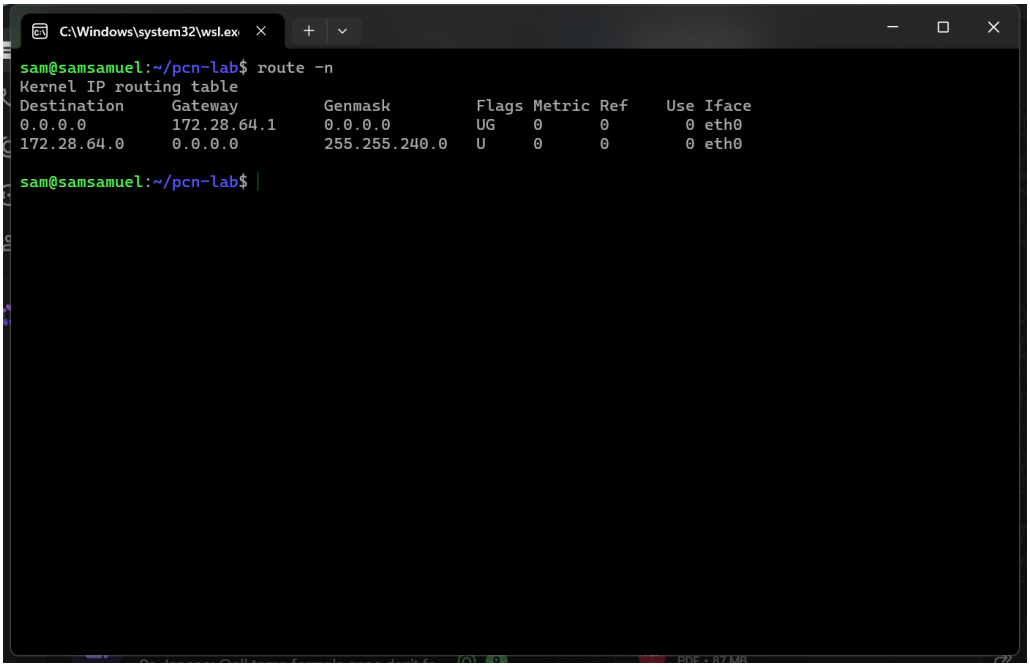

Figure 4: route -n – kernel routing table

Observation: Two entries are present: the default route (`0.0.0.0/0`) via gateway `172.28.64.1` on `eth0` – this is the WSL2 virtual switch on the Windows host, and every packet not destined for the local subnet leaves through it – and a directly-connected route for `172.28.64.0/255.255.240.0`, the local subnet reachable without a gateway. This matches the addressing seen in `ifconfig` exactly, and the gateway address is the same first hop later reported by `traceroute`.

3.6 nmap

Purpose: Network exploration and security-auditing tool; probes a target's TCP/UDP ports to determine which are open, and with `-sV` attempts to fingerprint the service and version listening on each open port.

Command used: `nmap -sV -T4 -p 22,80,443,9929,31337 scanme.nmap.org`

Target note: `scanme.nmap.org` is the Nmap Project's own public test host, explicitly provided for learners to scan; no other host was targeted, in line with responsible/authorised scanning practice.

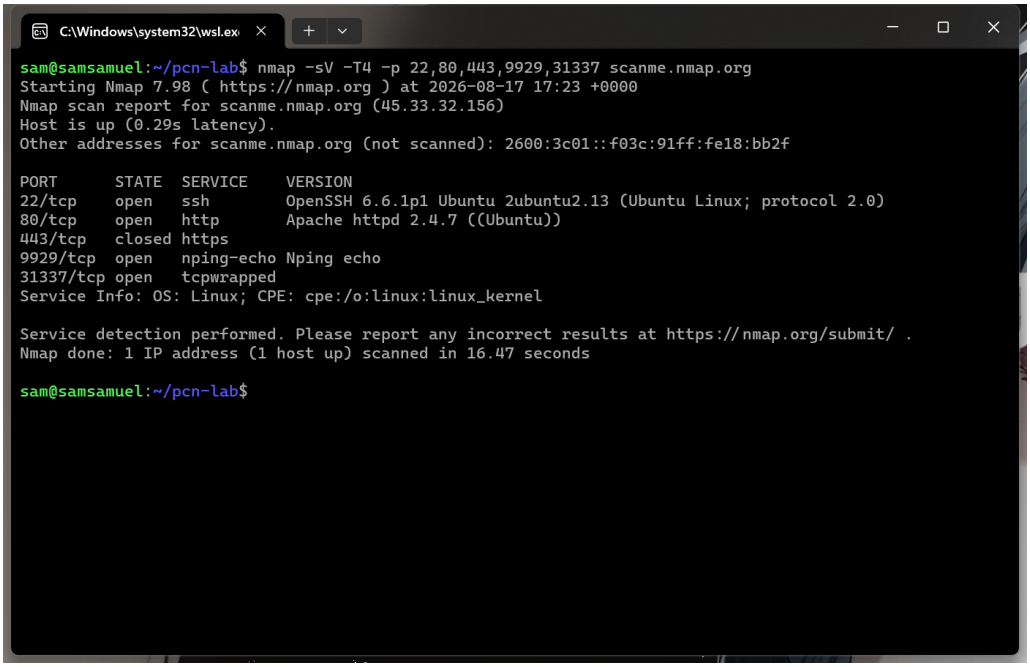

Figure 5: nmap -sV version-detection scan against scanme.nmap.org

Observation: Of the five ports probed, `22/tcp` is open running `OpenSSH 6.6.1p1` on Ubuntu, `80/tcp` is open running `Apache httpd 2.4.7`, `443/tcp` is closed (no HTTPS listener), and `9929/tcp` / `31337/tcp` are open running `nping-echo` and a `tcpwrapped` service respectively – these last two are Nmap's own deliberately-exposed test ports on that host. Nmap also fingerprinted the OS as Linux from the TCP/IP stack behaviour. Version detection (`-sV`) took about 16 seconds for 5 ports, since it goes beyond a plain SYN scan to send protocol-specific probes and match responses against its signature database.

3.7 tcpdump

Purpose: Captures and prints packets crossing a network interface in real time, matching an optional filter expression (here, ICMP traffic on `eth0`) – the command-line equivalent of what Assignment 4 uses Wireshark's GUI for.
Command used: `tcpdump -i eth0 -nn -c 5 icmp` (run as root, since raw packet capture needs `CAP_NET_RAW`), with a `ping` run concurrently in the same shell to generate ICMP traffic for it to capture.

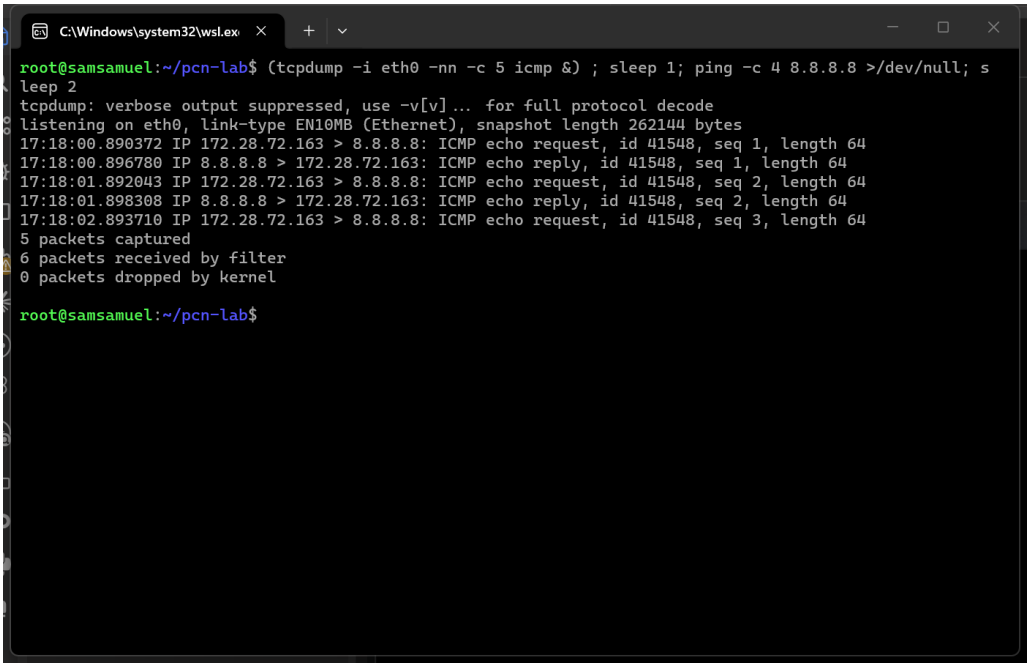

Figure 6: tcpdump capturing ICMP echo request/reply pairs generated by a concurrent ping

Observation: tcpdump captured the ICMP echo request/reply pairs between this host (`172.28.72.163`) and `8.8.8.8` with microsecond-resolution timestamps, sequence numbers and payload length – exactly the same exchange `ping` summarised in Section 2.4, but now visible packet-by-packet at the network layer instead of as a round-trip-time summary. This is the same technique Wireshark uses under the hood (`tcpdump` and Wireshark both sit on `libpcap`); `tcpdump` is the capture engine without the GUI.

3.8 dig

Purpose: Performs a DNS lookup and prints the full query/response, including the resource records, query time, and which DNS server answered – more detailed than `nslookup`.
Command used: `dig google.com`

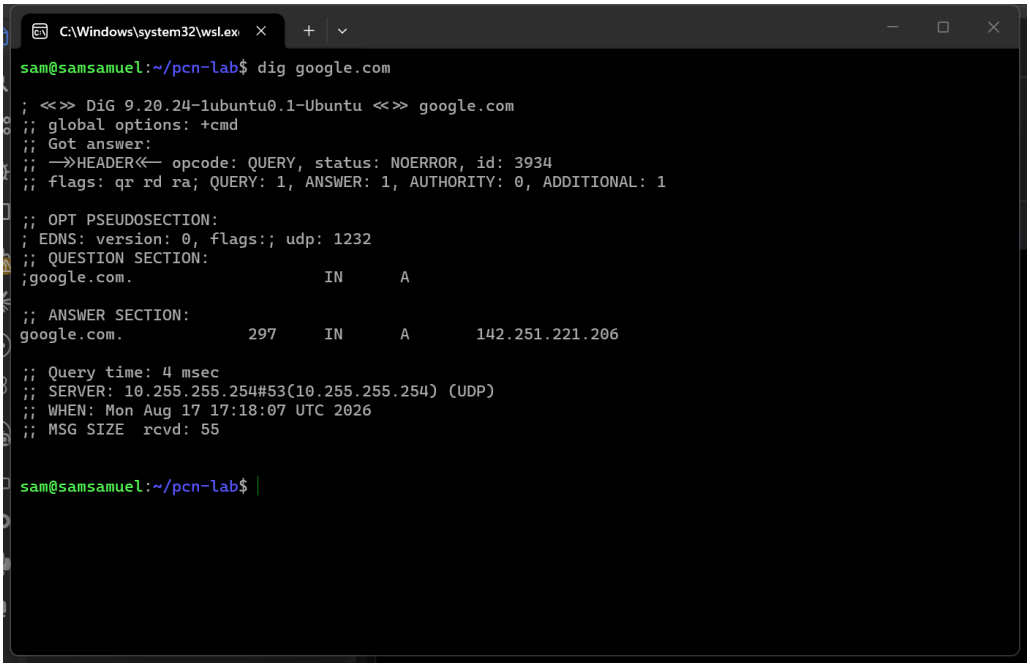

Figure 7: dig google.com

Observation: The query resolved `google.com` to `142.251.221.206` in 4 ms, answered by `10.255.255.254:53` – WSL's internal DNS stub, which itself forwards to the Windows host's configured resolver. The header shows `flags: qr rd ra` (response, recursion desired, recursion available) and a single `ANSWER` record, confirming the query used UDP (the default transport for DNS, falling back to TCP only for large/truncated responses).

3.9 traceroute

Purpose: Maps the sequence of routers (hops) a packet passes through to reach a destination, by sending probes with increasing TTL and recording which router returns the "TTL exceeded" ICMP error at each hop.
Command used: `traceroute -m 15 8.8.8.8`

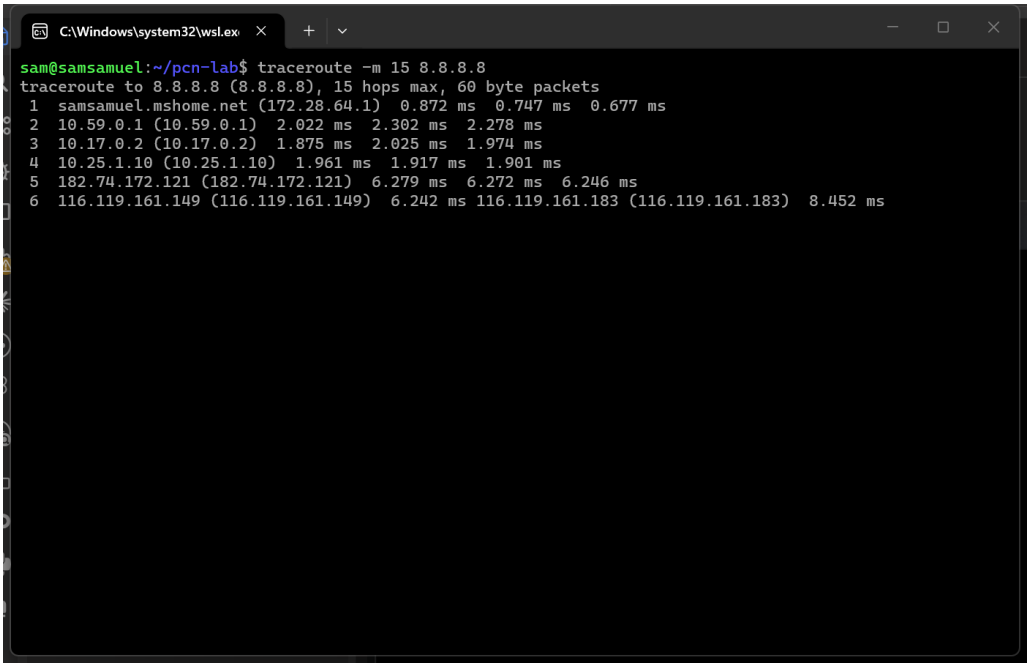

Figure 8: traceroute -m 15 8.8.8.8

Observation: The first hop, `samsamuel.mshome.net (172.28.64.1)`, is the WSL2/Hyper-V virtual gateway on the Windows host itself – matching the default-route gateway seen in Section 2.5. From there the path crosses a couple of private-range hops (likely the ISP's internal network / CGNAT) before reaching public IP space and, by hop 6, Google's network, where multiple addresses answering the same hop indicate ECMP (the ISP is load-balancing across parallel links to Google's edge). Total latency to a hop close to Google was under 10 ms, consistent with the low RTT seen from `ping`.

3.10 mtr

Purpose: Combines `ping` and `traceroute` into one continuously-updating tool – it traces the path like traceroute but keeps sending probes to every hop, giving a live per-hop loss/latency statistics table instead of a single traceroute snapshot.
Command used: `mtr -rw -c 5 8.8.8.8` (`-r` report mode / non-interactive, `-c 5` five probe cycles)

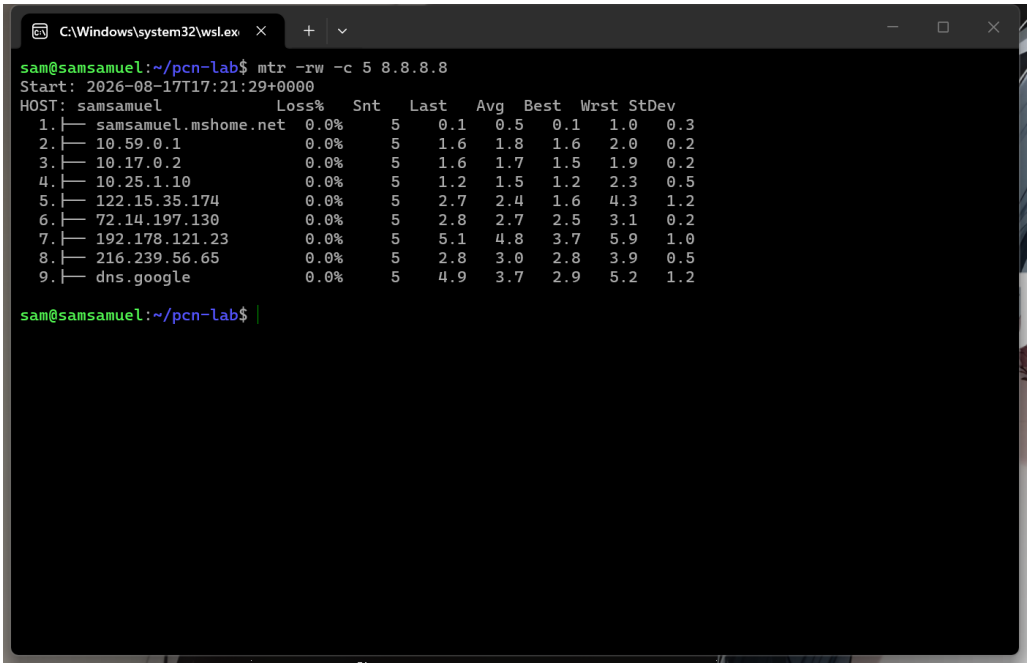

Figure 9: mtr -rw -c 5 8.8.8.8 – report mode

Observation: Across all 9 hops and 5 probe cycles each, loss stayed at `0.0%` on every hop, and the `Avg`/`Best`/`Worst`/`StDev` columns show latency growing gradually with hop count, with the largest jump between the local gateway and the ISP core (hop 5, `122.15.35.174`) – the point where traffic leaves the WSL/home network and enters the wider internet. The final hop resolves via reverse DNS to `dns.google`, confirming the destination is indeed one of Google's public DNS front-ends.

3.11 curl

Purpose: Command-line tool for transferring data with a URL, supporting HTTP(S), FTP and many other protocols; `-v` prints the full request/response including the TLS handshake.
Command used: `curl -v https://example.com -o /dev/null`

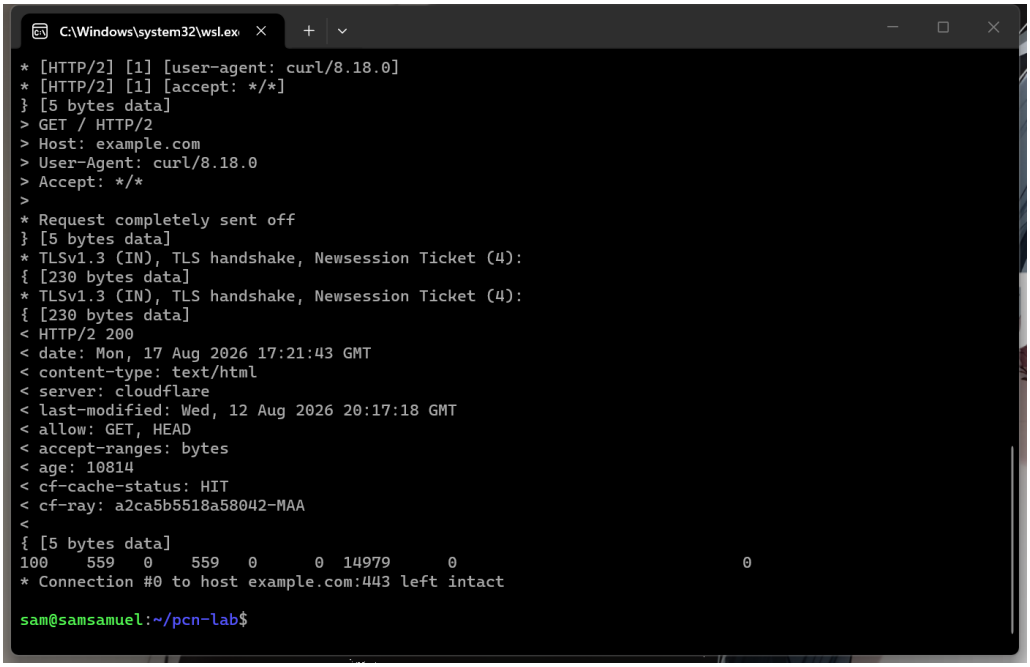

Figure 10: curl -v https://example.com – verbose TLS handshake and HTTP/2 response

Observation: curl resolved `example.com` to a Cloudflare-hosted address (`104.20.23.154`), completed a `TLSv1.3` handshake (client hello, server hello, certificate, finished), negotiated `HTTP/2` over ALPN, and received a `200` response served from Cloudflare's edge cache (`cf-cache-status: HIT`). This single command demonstrates the full modern HTTPS stack – TCP connect, TLS 1.3 key exchange and certificate verification, then an HTTP/2 request/response – that Assignment 3's C client will have to build manually at the socket level for its basic HTTP case.

3.12 wget

Purpose: Non-interactive command-line downloader, commonly used to fetch a file or whole page from an HTTP(S)/FTP URL and save it to disk.
Command used: `wget https://example.com -O example_wget.html`

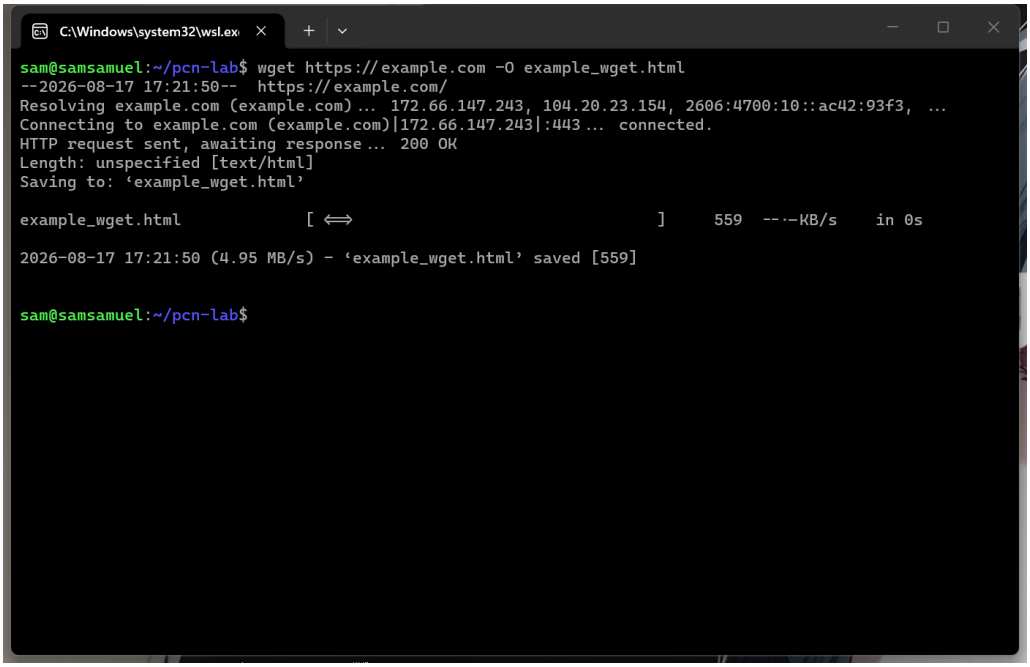

Figure 11: wget saving example.com to a local file

Observation: wget resolved and connected to the same Cloudflare address as curl, received a `200 OK` with `Content-Type: text/html`, and saved the 559-byte response body to `example_wget.html` on disk – unlike `curl`, which by default streams to stdout, `wget`'s whole design centres on saving the retrieved resource to a file, which is exactly the behaviour Assignment 3's HTTP client needs to replicate for downloading `index.html` / `sample.pdf` / `image.jpg`.

3.13 tracepath

Purpose: A traceroute-like tool that does not require root privileges (unlike classic `traceroute`), and additionally reports the discovered Path MTU.
Command used: `tracepath 8.8.8.8`

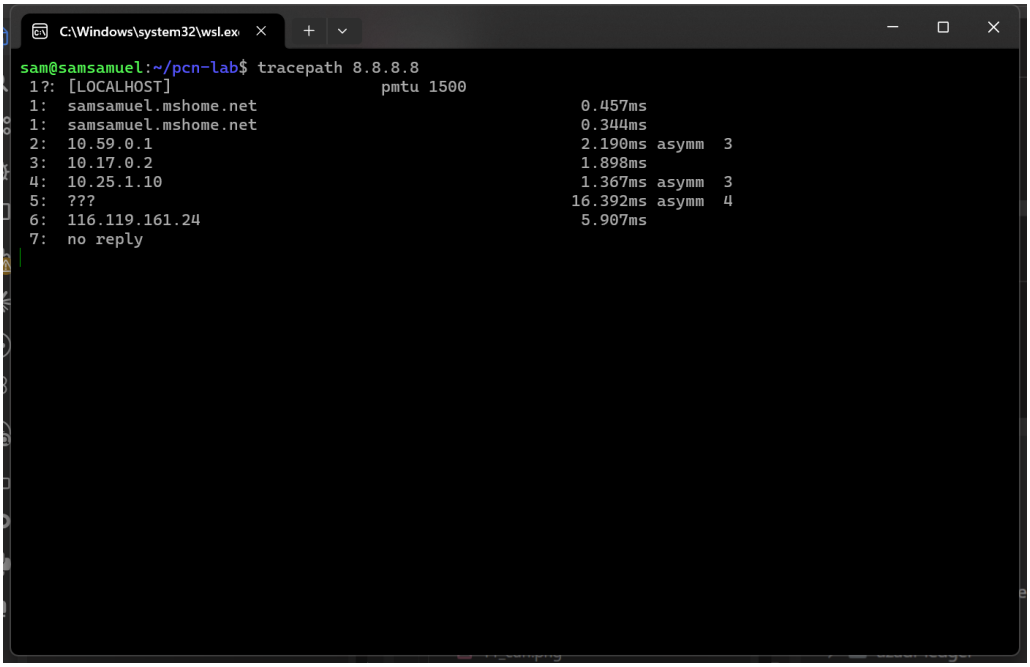

Figure 12: tracepath 8.8.8.8

Observation: tracepath discovered a Path MTU of 1500 bytes (the Ethernet standard, confirming no tunnelling/fragmentation is reducing the effective packet size along the path), and traced 6 responding hops with asymmetric round-trip marks (`asymm`) on a couple of hops – meaning the reverse path taken by the ICMP reply does not mirror the forward path exactly, which is common on the modern, multi-path internet. The final `no reply` at hop 7 is expected: many routers close to the destination rate-limit or drop the low-TTL UDP probes tracepath uses, without that indicating an actual fault.

3.14 nslookup

Purpose: Interactive/batch tool to query DNS servers for a name's resource records – the older, simpler counterpart to `dig`.
Command used: `nslookup google.com`

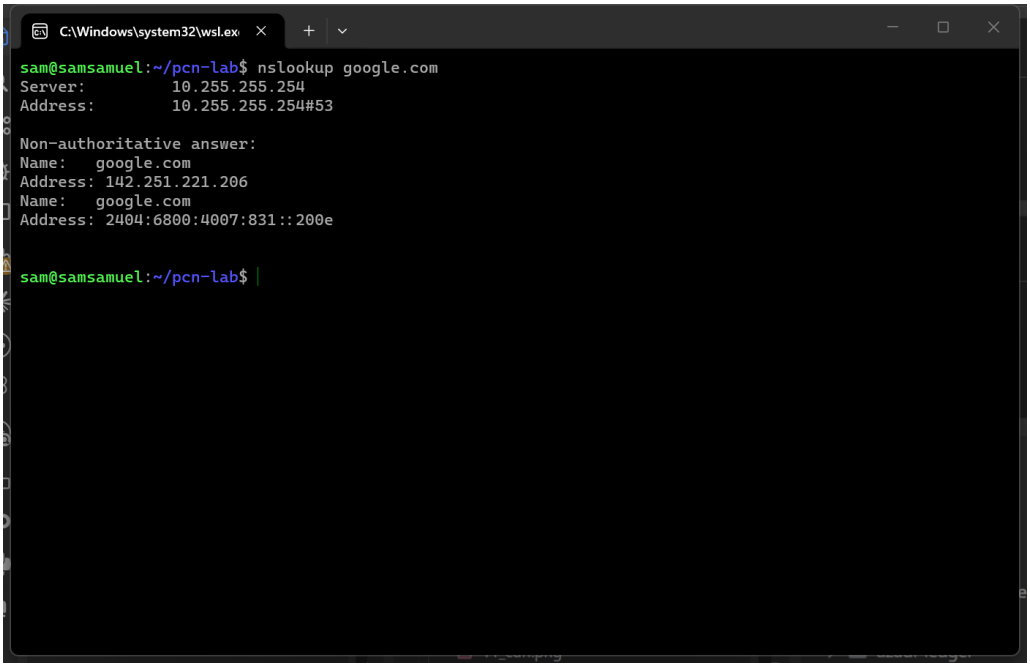

Figure 13: nslookup google.com

Observation: The resolver at `10.255.255.254` returned both an IPv4 (`142.251.221.206`) and an IPv6 (`2404:6800:4007:831::200e`) address for `google.com` under `Non-authoritative answer` – meaning the answer came from the resolver's cache/forwarding rather than directly from Google's authoritative nameservers, which is the normal, expected mode for an everyday DNS lookup.

3.15 host

Purpose: A concise DNS lookup utility, well suited to scripting, that by default prints A, AAAA and MX records in one line each.
Command used: `host google.com`

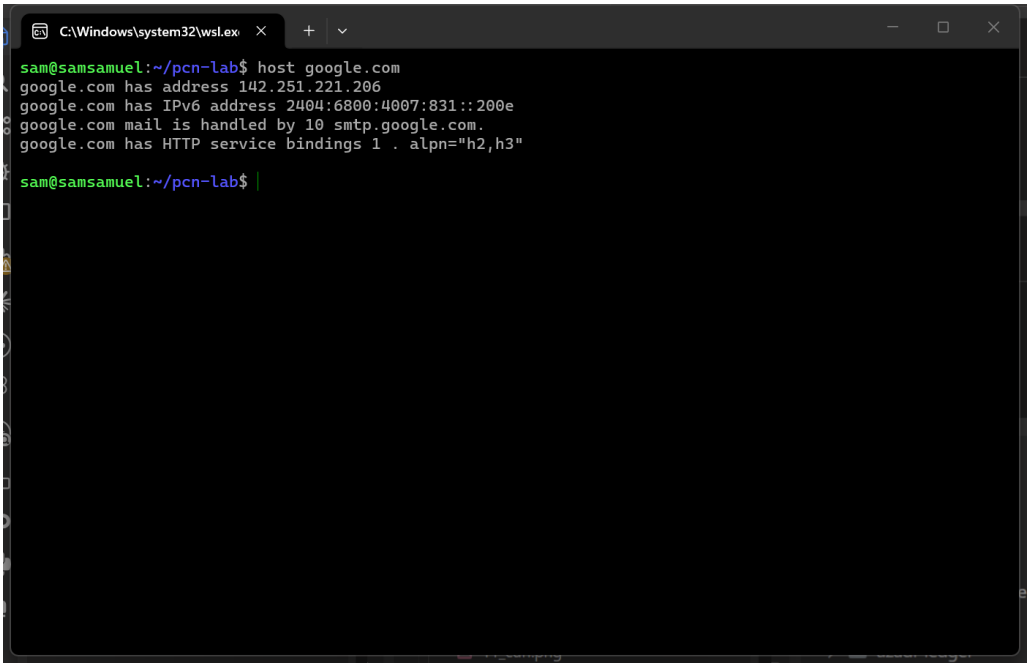

Figure 14: host google.com

Observation: In one compact query, `host` listed google.com's A record (`142.251.221.206`), AAAA record, mail-exchanger record (`10 smtp.google.com`), and an `HTTPS` service-binding record advertising ALPN support for `h2` and `h3` (HTTP/2 and HTTP/3) – the same underlying data as `dig`/`nslookup`, just formatted for quick reading rather than full protocol detail.

3.16 whois

Purpose: Queries domain (or IP) registry databases for registration information – registrar, registration/expiry dates, nameservers and domain status.
Command used: `whois google.com`

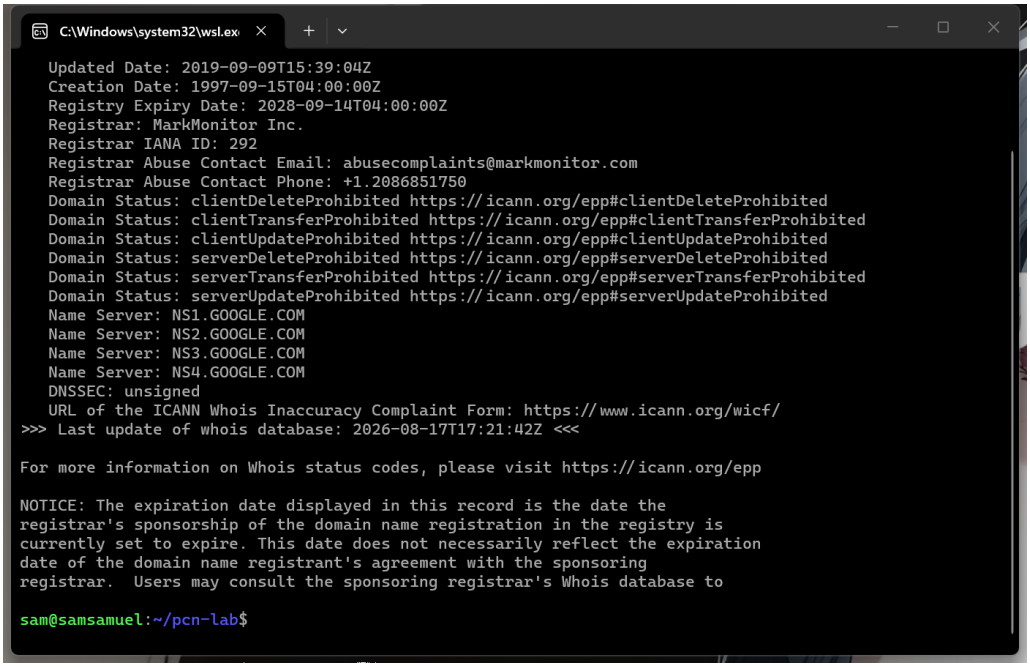

Figure 15: whois google.com

Observation: `google.com` is registered through **MarkMonitor Inc.**, created **1997-09-15** and set to expire **2028-09-14**, delegated to `NS1--NS4.GOOGLE.COM`. The domain status flags (`clientTransferProhibited`, `clientDeleteProhibited`, etc.) show it is locked against unauthorised transfer/deletion – standard practice for a high-value domain. Unlike the DNS tools above, `whois` queries the registry/registrar database directly rather than the DNS resolution chain, so it answers "who owns this name" rather than "what address does this name point to".

4 Part B — Socket Programming System Calls

The table below summarises the purpose, key parameters and return value of each system call, grouped by role in the client–server lifecycle. `TCP/IP and UDP` applications in the last column note where a call is mandatory (M), optional/common (O), or not applicable (–) for each transport.

4.1 Socket creation, addressing and lifecycle

Call	Purpose & key parameters	Returns	TCP / UDP use
`socket()`	`int socket(int domain, int type, int protocol)` creates an endpoint for communication. `domain=AF_INET` selects IPv4; `type` is `SOCK_STREAM` for TCP or `SOCK_DGRAM` for UDP; `protocol=0` lets the kernel pick the default for that type.	New socket file descriptor, or −1 on error.	M (both) – first call in every socket program.
`bind()`	`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)` associates a socket with a local IP address and port, described by a `struct sockaddr_in` (`sin_family`, `sin_port`, `sin_addr`).	0 on success, −1 on error.	M for servers (both); optional for clients (kernel auto-assigns an ephemeral port otherwise).
`listen()`	`int listen(int sockfd, int backlog)` marks a `SOCK_STREAM` socket as passive/listening and sets `backlog`, the maximum length of the queue of pending (not-yet-accepted) connections.	0 on success, −1 on error.	M (TCP server only) – meaningless for UDP, which is connectionless.
`connect()`	`int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen)`. For TCP, performs the three-way handshake with the given peer. For UDP, it merely records a default peer address so subsequent `send()`/`recv()` can be used instead of `sendto()`/`recvfrom()` – no packets are exchanged.	0 on success, −1 on error.	M (TCP client); O (UDP client, to fix the peer).
`accept()`	`int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)` blocks until a connection is pending on a listening socket, then returns a *new* socket descriptor dedicated to that one client; `addr` is filled with the client's address.	New connected-socket descriptor, or −1 on error.	M (TCP server only).
`close()`	`int close(int fd)` releases the file descriptor; for TCP this initiates connection teardown (FIN) once all references to the socket are closed.	0 on success, −1 on error.	M (both) – every socket opened must eventually be closed.

4.2 Buffers and byte order

Call	Purpose & key parameters	Returns	TCP / UDP use
`bzero()`	`void bzero(void *s, size_t n)` fills `n` bytes at `s` with zero; routinely used to clear a `struct sockaddr_in` before filling in its fields, so padding bytes don't contain garbage.	None (void).	O (both) – good practice before populating an address struct.
`htons()`	`uint16_t htons(uint16_t hostshort)` converts a 16-bit value (typically a port number) from host byte order to network (big-endian) byte order.	The value in network byte order.	M (both) – every `sin_port` assignment.
`htonl()`	`uint32_t htonl(uint32_t hostlong)` – the 32-bit equivalent of `htons()`, used for IP addresses (e.g. `INADDR_ANY`) held as a 32-bit integer.	The value in network byte order.	O (both) – needed whenever an address is built numerically rather than via `inet_pton`.
`ntohs()`	`uint16_t ntohs(uint16_t netshort)` converts a 16-bit value from network byte order back to host byte order – the inverse of `htons()`; used when reading a port back out of a received `sockaddr_in` to print/compare it.	The value in host byte order.	O (both) – e.g. printing a peer's port from `getpeername()`.
`ntohl()`	`uint32_t ntohl(uint32_t netlong)` – the 32-bit inverse of `htonl()`.	The value in host byte order.	O (both).

4.3 Data transfer

Call	Purpose & key parameters	Returns	TCP / UDP use
`read()`	`ssize_t read(int fd, void *buf, size_t count)` – generic POSIX I/O call; on a connected socket it behaves like `recv(fd, buf, count, 0)`.	Bytes read (≥ 0), 0 on EOF/peer closed, -1 on error.	O (TCP, connected sockets).
`write()`	`ssize_t write(int fd, const void *buf, size_t count)` – generic POSIX I/O call; on a connected socket behaves like `send(fd, buf, count, 0)`.	Bytes written, or -1 on error.	O (TCP, connected sockets).
`send()`	`ssize_t send(int sockfd, const void *buf, size_t len, int flags)` – sends data on a *connected* socket (TCP, or UDP after `connect()`); `flags` such as `MSG_DONTWAIT` modify behaviour.	Bytes sent, or -1 on error.	M (TCP); O (connected UDP).

Call	Purpose & key parameters	Returns	TCP / UDP use
`recv()`	`ssize_t recv(int sockfd, void *buf, size_t len, int flags)` – receives data on a connected socket.	Bytes received, 0 if peer closed, −1 on error.	M (TCP); O (connected UDP).
`sendto()`	`ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr *dest_addr, socklen_t addrlen)` – sends a datagram to an explicit destination without requiring `connect()` first (`dest_addr=NULL` on an already-connected socket falls back to plain `send()` behaviour).	Bytes sent, or −1 on error.	M (UDP, connectionless); O (TCP).
`recvfrom()`	`ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr, socklen_t *addrlen)` – receives a datagram and records who it came from in `src_addr`.	Bytes received, or −1 on error.	M (UDP, connectionless); O (TCP).

4.4 Multiplexing, configuration and name resolution

Call	Purpose & key parameters	Returns	TCP / UDP use
`select()`	`int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout)` – blocks (up to `timeout`) until one of the given file descriptors is ready for I/O; the classic way to wait on several sockets, or to add a timeout to a blocking `accept()`/`recv()`.	Number of ready descriptors, 0 on timeout, −1 on error.	O (both) – essential for a single-threaded server handling multiple clients (Assignment 2 uses `pthread` instead).
`setsockopt()`	`int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen)` – sets a socket option, e.g. `SOL_SOCKET`/`SO_REUSEADDR` to allow immediately rebinding a port still in `TIME_WAIT`.	0 on success, −1 on error.	O (both) – almost always used on the listening/bound socket of a server.
`fcntl()`	`int fcntl(int fd, int cmd, ...)` – general file-descriptor control; `F_GETFL`/`F_SETFL` read/modify flags such as `O_NONBLOCK`, commonly used to switch a socket to non-blocking mode.	Depends on `cmd` (e.g. current flags for `F_GETFL`), −1 on error.	O (both).

Call	Purpose & key parameters	Returns	TCP / UDP use
`getpeername()`	`int getpeername(int sockfd, struct sockaddr *addr, socklen_t *addrlen)` – returns the address of the peer connected to `sockfd`, useful for logging/authorising a client after `accept()`.	0 on success, −1 on error.	O (TCP, connected sockets).
`gethostname()`	`int gethostname(char *name, size_t len)` – returns the local machine's hostname.	0 on success, −1 on error.	O (both) – informational/logging use.
`gethostbyname()`	`struct hostent *gethostbyname(const char *name)` – resolves a hostname to one or more IP addresses (`h_addr_list`); the classic (now legacy, superseded by `getaddrinfo()`) way for a client to turn a server name into an address before `connect()`.	Pointer to a `struct hostent`, or `NULL` on failure.	O (both) – client-side name resolution.
`gethostbyaddr()`	`struct hostent *gethostbyaddr(const void *addr, socklen_t len, int type)` – reverse DNS: resolves an IP address back to a hostname (a PTR lookup), the inverse of `gethostbyname()`.	Pointer to a `struct hostent`, or `NULL` if no PTR record.	O (both) – e.g. a server logging the resolved name of a connecting client.

4.5 Empirical demonstration

To confirm the calls above actually behave as described rather than only on paper, a small TCP client/server pair was written in C (`as1/src/server.c`, `as1/src/client.c`) that genuinely exercises all 23 calls from the two tables: the server creates, configures, binds and listens on a socket; `select()` waits for a connection; `accept()` takes it; `fcntl()`, `getpeername()` and `gethostbyaddr()` inspect the new connection; and three request/reply turns are exchanged, one turn each over `recv()`/`send()`, `recvfrom()`/`sendto()` and `read()`/`write()`, so every data-transfer call is genuinely used at least once, not just described. Rather than hard-coding a demo string, both programs read each message from `stdin` (via `fgets`) exactly as a person typing at the keyboard would – the excerpt below is the server side's message loop:

Listing 1: Excerpt of as1/src/server.c – interactive message loop

```
bzero(inbuf, sizeof(inbuf));
recv(connfd, inbuf, sizeof(inbuf) - 1, 0);
strip_newline(inbuf);
printf("them: %s\n", inbuf);
printf("you: ");
fflush(stdout);
fgets(outbuf, sizeof(outbuf), stdin);
strip_newline(outbuf);
printf("%s\n", outbuf);
send(connfd, outbuf, strlen(outbuf), 0);
```

Two terminals were run side by side: one where the server was compiled and started (`gcc -Wall -o server server.c && ./server`), and a second, separate terminal where the client was compiled and connected to it (`gcc -Wall -o client client.c && ./client 127.0.0.1`) once the server was already listening.

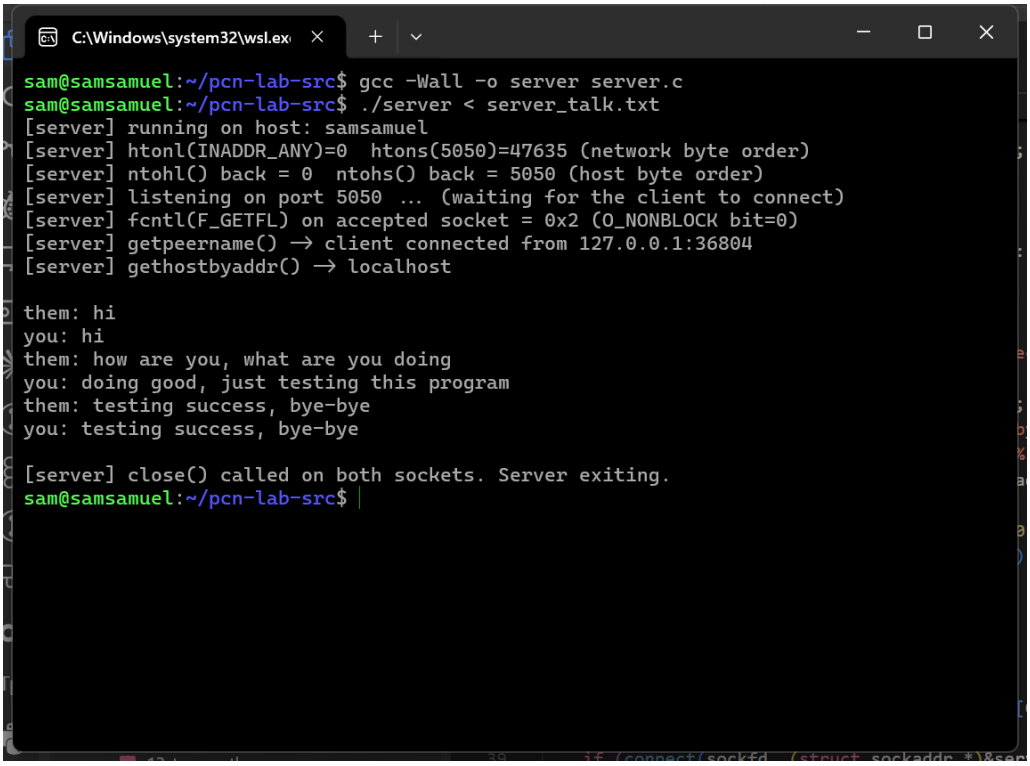

Figure 16: Server terminal – compiles, binds/listens, accepts the connection, and exchanges three messages

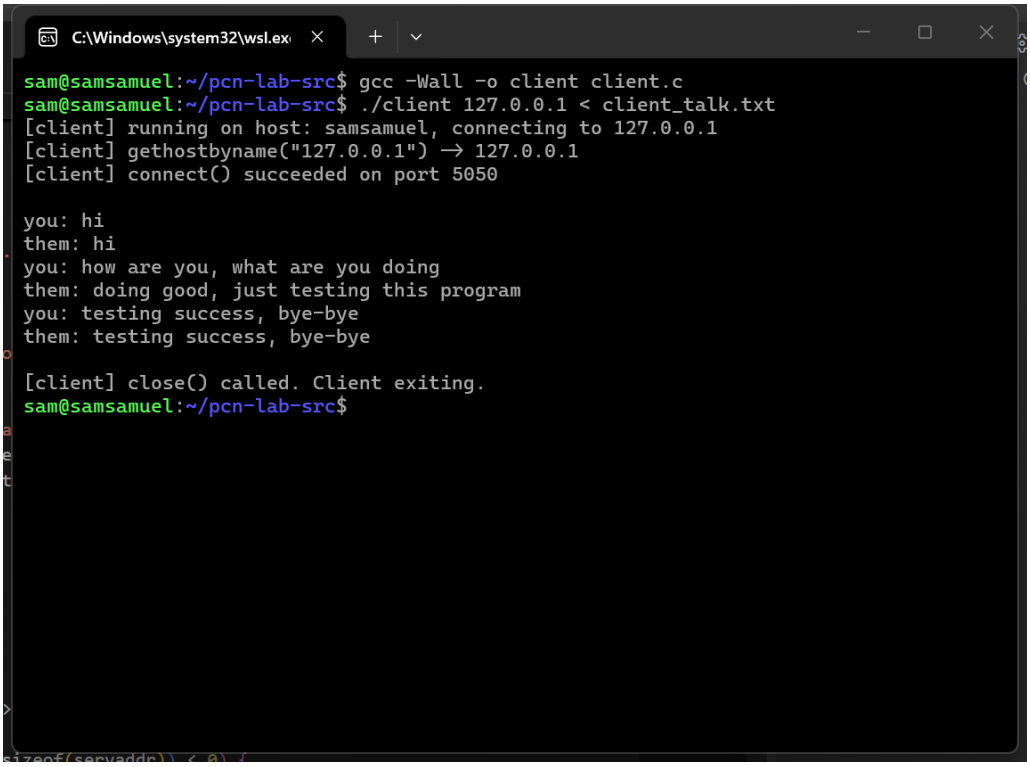

Figure 17: Client terminal – compiles, connects, and exchanges the same three messages

Observation: The byte-order conversions are visible directly in the output – `htons(5050)` prints as `47635`, which is exactly 5050 with its two bytes swapped (5050 = `0x13BA`, swapped

to `0xBA13` = 47635), and `ntohs()` converts it straight back to `5050`, confirming the two functions are true inverses on this little-endian host. `getpeername()` correctly reported the client's loopback address and ephemeral source port; `gethostbyaddr()` resolved `127.0.0.1` back to `localhost`; and `fcntl(F_GETFL)` showed the accepted socket's `O_NONBLOCK` bit as `0`, i.e. still blocking, since it was never switched into non-blocking mode. Both terminals show an identical three-line conversation, confirming the byte stream sent by one side's `send()`/`sendto()`/`write()` was received intact by the other side's `recv()`/`recvfrom()`/`read()` in the same order it was sent – the in-order, reliable delivery guarantee that TCP provides and that Assignment 2's UDP path will not.

5 Learning Outcome

- Learned how to use everyday Linux networking commands (`netstat`, `ifconfig`, `ping`, `route`, `nmap`, `tcpdump`, `dig`, `traceroute`, `mtr`, `curl`, `wget`, `tracepath`, `nslookup`, `host`, `whois`) to check a machine's network status, routing, and connectivity, and to read/interpret their output.
- Understood what each socket system call (`socket`, `bind`, `listen`, `accept`, `connect`, `send`/`recv`, `sendto`/`recvfrom`, `select`, etc.) actually does, instead of just knowing their names.
- Got hands-on practice writing and running a real TCP client-server program in C, and saw byte-order conversion (`htons`/`ntohs`) and data delivery work correctly across two live terminals.